第 1 周：优化的边界

计算复杂性与问题的本质难度

目录

1 课程总览 **3**

2 考核方式 **8**

3 优化问题的一般形式 **9**

4 什么是”计算”？ **10**

4.1 图灵机：计算的形式化 10

5 图灵机一定会停机吗？ **14**

5.1 停机问题 14

5.2 数学证明就是停机 15

6 简单规则，复杂行为 **20**

7 哥德尔不完备性定理 **25**

8 P 与 NP：可行计算的边界 **26**

9 NP-Complete 与归约 **27**

10 为什么 SAT 能表达一切 NP 问题？ **31**

10.1 3-SAT：恰到好处的复杂度 32

10.2 2-SAT：为什么突然变简单了？ 34

11 为什么图灵机是「计算」的标准？ **35**

11.1 模型 1：图灵机 36

11.2 模型 2：λ 演算 36

11.3 模型 3：递归函数 37

11.4 模型 4：Post 系统 38

11.5 四种模型的统一 39

12 上限-下限框架 **40**

12.1 完整的复杂度层级 41

12.2 各复杂度类的著名问题 41

13 GPU 改变复杂度观念 **46**

14 本周总结 **48**

15 实验 **50**

15.1 实验 1a：TSP - 结构的力量 50

15.2 实验 1b：GPU 并行 vs CPU 顺序 57

16 思考题 **61**

1 课程总览

这门课想做什么

不是一门传统的优化理论课（那样的课很多）
而是带大家思考：

> **现代优化**对**机器学习**和相关问题有哪些思考？
>
> **GPU 时代**对**优化算法设计**带来了什么改变？

你不需要：

- 记住所有定理的证明细节
- 成为每个子领域的专家

你需要：

- 建立对优化问题的**直觉**——什么是难的，什么是容易的
- 理解几个**核心思想**——它们反复出现
- 感受**理论与实践的差距**——以及如何弥合

课程的核心哲学

一、上限-下限框架

科研 = 发现问题的**结构**，开发**针对性方法**

二、GPU 范式转变

经典思维	GPU 时代
$O(n \log n)$ 最优	并行 $O(n^2)$ 可能更快
减少 FLOPS	减少 IO（内存访问）
二阶方法	一阶 + 简单预条件
小 batch 精确	大 batch 高效

三、没有银弹

不存在”万能优化算法”。每个问题都有其**本质难度**，我们要学会**识别结构**、**选择工具**。

16 周课程地图

周	主题	核心
1	**优化的边界**（计算复杂性）	什么是难的？← 本周（P/NP/近似/下界直觉：为何“优化器并不能解决一切”）
2	凸优化与 KKT	对偶的力量（KKT= 约束优化的“语法”；为后续 TRPO/PPO 做铺垫）
3	LP 对偶与近似算法	用对偶证明近似比（Primal-dual、LP rounding、局部搜索）
4	算法设计范式与 GPU 化	分治/DP/贪心（并行化/算子融合/内存带宽：为何“实现细节”决定优化速度）
5	梯度法与预条件	预条件与条件数：GD/SGD vs Heavy-ball/Momentum/Nesterov； 对角预条件：AdaGrad/RMSProp/Adam；矩阵/结构预条件：Natural Grad, K-FAC, Shampoo/（SOAP）
6	流形优化	流形结构（Riemannian GD；Stiefel/Orthogonal：与 whitening/orthogonalization 的连接）
7	随机化算法	随机化让不可能变可能（随机投影/Sketch/SVD 近似：为低秩预条件埋伏笔）
8	随机优化	SGD/Adam/大 Batch：AdamW（解耦权重衰减）；LARS/LAMB（大 batch 缩放）； 内存友好：Adafactor（因子化二阶矩）；新范式：Lion（符号动量）
9	深度学习优化理论	NTK/隐式正则化（SGD 噪声、sharp vs flat、学习率/批大小与泛化）
10	变分法与最优控制	泛函优化/Neural ODE（连续时间视角：动量 = 二阶系统；预条件 = 度量选择）
11	在线优化与 Regret	Regret框架（Mirror Descent/Follow-the-Regularized-Leader：连接 AdaGrad/Adam 的解释）
12	MDP 与 Tabular RL	Bellman/值迭代（优化角度：固定点迭代、收敛与 contraction）
13	深度 RL 与策略优化	Fisher 几何/预条件：Natural Policy Gradient / TRPO； 近似：K-FAC-style Fisher；工程：AdamW 作为 baseline；可选：Cautious wrapper（提升稳定性）
14	PPO——约束优化的巅峰	从 KKT 到 RLHF（clipping/penalty 视角；约束 → 近端 → 稳定训练）
15	分布式优化	ADMM/通信压缩（Distributed Shampoo/ZeRO/梯度压缩；大规模训练的 optimizer-state 成本）
16	大规模优化与总结	FlashAttention/总结（系统 × 优化器：算子/通信/内存； 前沿：Whitening/Spectral（Muon/low-rank precond./spectral shrinkage；以及 Meta-optimizer 方向）

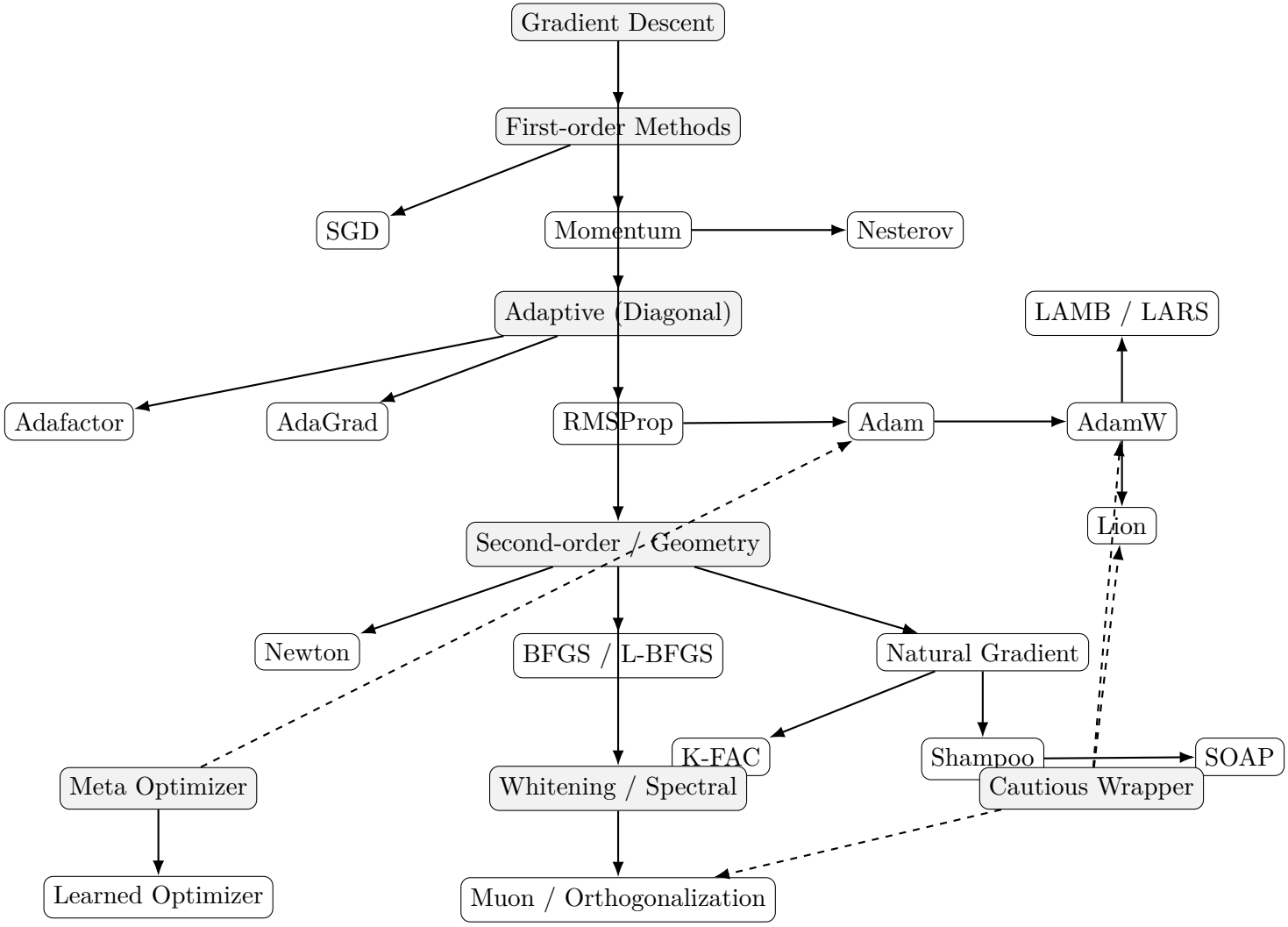


图 1: Evolution of Optimization Methods

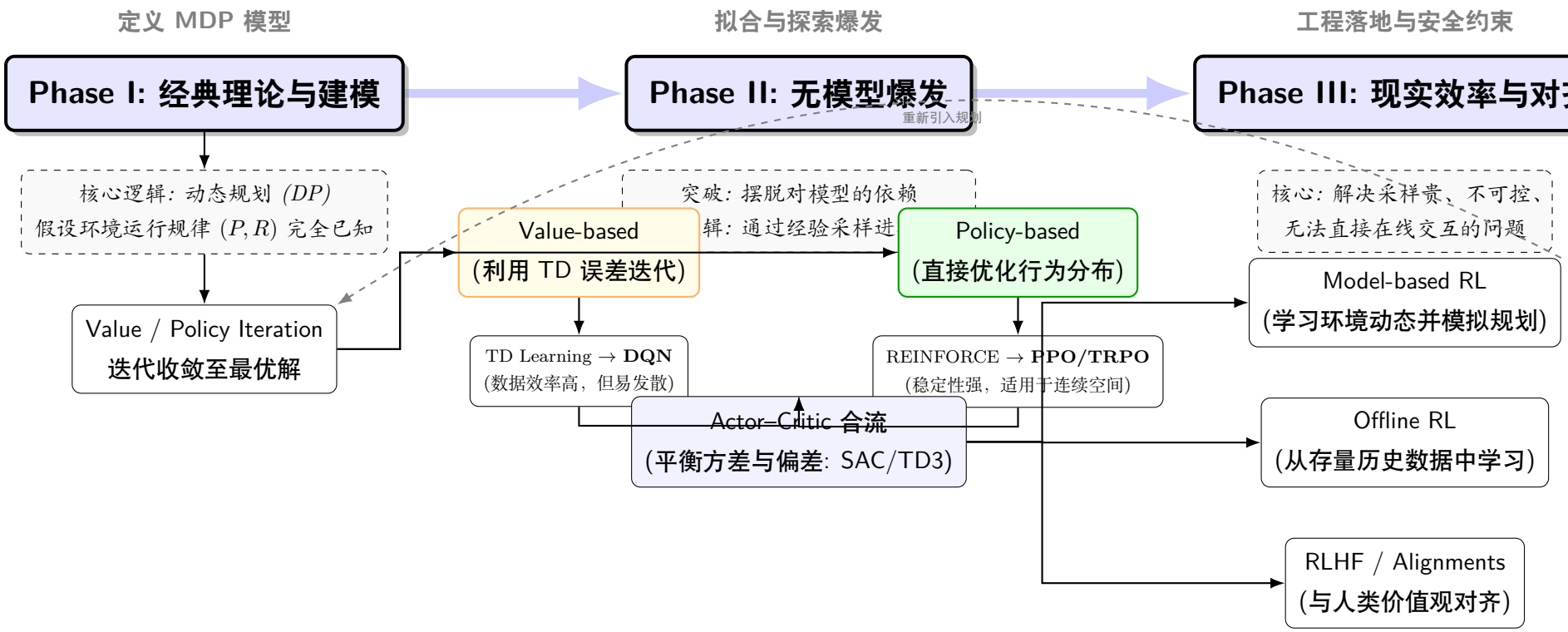


图 2: Evolution of Reinforcement Learning Methods

五条暗线

贯穿整个课程的五个核心思想：

1. **对偶性**（周 2,3,10,14,15）

$$\text{KKT} \to \text{LP 对偶} \to \text{伴随方法} \to \text{PPO} \to \text{ADMM}$$

2. **预条件**（周 5,8,13,16）

$$\text{牛顿法} \to \text{Adam} \to \text{Fisher 信息} \to \text{K-FAC}$$

3. **Regret**（周 5,8,9,11,12）

$$\text{优化器} \to \text{在线学习} \to \text{Bandit} \to \text{MDP}$$

4. **随机化**（周 7,8,9）

$$\text{随机矩阵乘法} \to \text{SGD} \to \text{Hessian 估计}$$

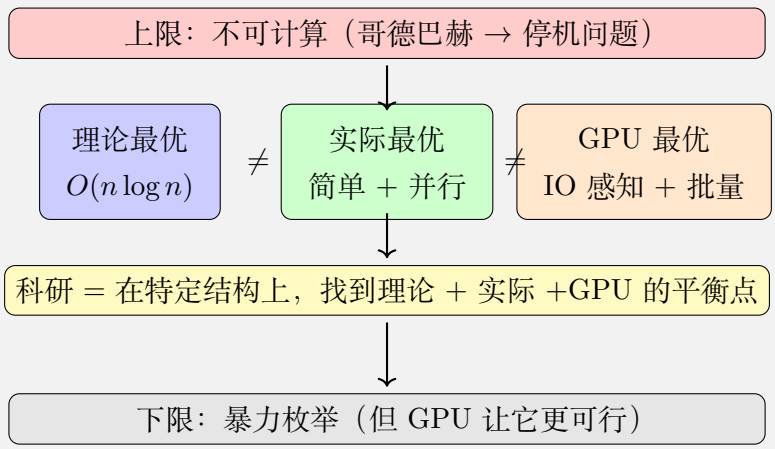
5. **流形结构**（周 6,13,14）

$$\text{Stiefel 流形} \to \text{Fisher 几何} \to \text{muon} \to \text{信任域}$$

第六条暗线：GPU 改变一切

每周实验（20-35 分钟）：亲身体验理论与实践的差距

课程结构图



2 考核方式

成绩构成

项目	分值	说明
平时小测	40 分	课堂小测，题目类似作业
期末考试	60 分	闭卷，题目类似作业
代码贡献	+10 分	完善课程例子（加分项）

重要规则

小测保护机制：
如果你的**小测总分 < 考试得分率 × 40**，则按考试得分率重新计算小测分
例子：

- 小测得了 20/40，考试得了 48/60（80%）
- 小测分 $= \max(20, 40 \times 80\%) = \max(20, 32) = 32$

含义：

- 你可以**直接来考试**，考好了小测分会被”拉上来”
- 平时认真学，小测考好，双保险

作业与代码贡献

作业：

- 每节课都有作业
- **不计分**，但小测/考试题和作业非常类似

代码贡献加分（最多 +10 分）：

- 完善课程的代码示例
- 实现有趣的可视化
- 发现并修复 bug
- 添加有教学价值的实验

标准：代码对大家讨论学习有帮助
注：解释权归授课教师

核心结论：学习建议

1. 抓住核心思想

不要纠结于每个证明的细节。理解**为什么**比记住**怎么做**更重要。

2. 建立联系

注意五条暗线如何反复出现。优化的核心思想其实不多。

3. 问"为什么"

- 为什么这个方法有效?
- 什么时候会失效?
- GPU 上会怎样?

3 优化问题的一般形式

统一框架

优化问题的一般形式:

$$\min_{\boldsymbol{x}} f(\boldsymbol{x}) \quad \text{s.t.} \quad \boldsymbol{x} \in \mathcal{X}$$

- $\boldsymbol{x}$：决策变量（可以是向量、矩阵、函数...）
- f：目标函数（要最小化的东西）
- $\mathcal{X}$：可行域（约束条件）

我们想要什么？

- 找到最优解 $\boldsymbol{x}^*$
- 或者：找到"足够好"的解
- 关键问题：**能不能做到？要多久？**

优化问题的例子

机器学习:

$$\min_{\theta} \frac{1}{n} \sum_{i=1}^{n} \ell(f_\theta(x_i), y_i) + \lambda R(\theta)$$

推荐系统:

$$\min_{U,V} \sum_{(i,j)\in\Omega} (R_{ij} - U_i^\top V_j)^2 \quad \text{s.t.} \quad \operatorname{rank}(UV^\top) \le k$$

资源分配：

$$\max_{\boldsymbol{x}} \boldsymbol{c}^\top \boldsymbol{x} \quad \text{s.t.} \quad A\boldsymbol{x} \le \boldsymbol{b},\ \boldsymbol{x} \ge 0$$

强化学习：

$$\max_{\pi} \mathbb{E}_{\tau\sim\pi}\left[\sum_{t=0}^{\infty} \gamma^t r_t\right]$$

共同点：都是优化问题！但**结构**非常不同。

4 什么是”计算”？

计算的本质问题

问题：什么是”算法”?
直觉答案：一系列明确的步骤，机械地执行，最终给出答案
但是：

- ”明确”是什么意思?
- ”机械地”怎么定义?
- 一定能给出答案吗?

图灵的回答（1936）：用一个极简的机器来定义”计算”

4.1 图灵机：计算的形式化

图灵机的组成

图灵机是一个极其简单但强大的计算模型，由三部分组成：

三要素：

1. **纸带**：无限长，每格存一个符号（0 或 1）
2. **读写头**：指向当前格子，可读取、写入、左移、右移
3. **状态**：有限个状态 $q_1, q_2, \ldots$（显示在读写头内）

转移函数：图灵机的”程序”

图灵机的行为由**转移函数** δ 定义：

$$\delta(\text{当前状态}, \text{读到的符号}) = (\text{写入符号}, \text{移动方向}, \text{新状态})$$

例子：考虑以下 3 状态图灵机

- 字母表：$\{0, 1\}$
- 状态：q_1（起始），q_2，q_3（停机）

转移规则：

$$\begin{aligned} \delta(q_1, 0) &= (1, R, q_2) & \delta(q_1, 1) &= (1, R, q_1) \\ \delta(q_2, 0) &= (0, L, q_3) & \delta(q_2, 1) &= (1, R, q_2) \\ \delta(q_3, 0) &= (0, H, q_3) & \delta(q_3, 1) &= (0, H, q_3) \end{aligned}$$

其中 $R =$ 右移，$L =$ 左移，$H =$ 停机

转移表：更直观的表示

同样的规则可以用**转移表**表示：

	0	**1**
q_1	$1\,R\,q_2$	$1\,R\,q_1$
q_2	$0\,L\,q_3$	$1\,R\,q_2$
q_3	$0\,H\,q_3$	$0\,H\,q_3$

读法：当前状态 q_1，读到 0 $\Rightarrow$ 写 1，右移，转到 q_2

这个机器做什么？

- q_1：向右扫描，跳过所有 1；遇到 0 时写成 1，进入 q_2
- q_2：继续向右扫描，跳过所有 1；遇到 0 时左移，进入 q_3
- q_3：把当前格子写 0，停机

图灵机完整执行过程（第一部分：$t = 0$ 到 $t = 7$）

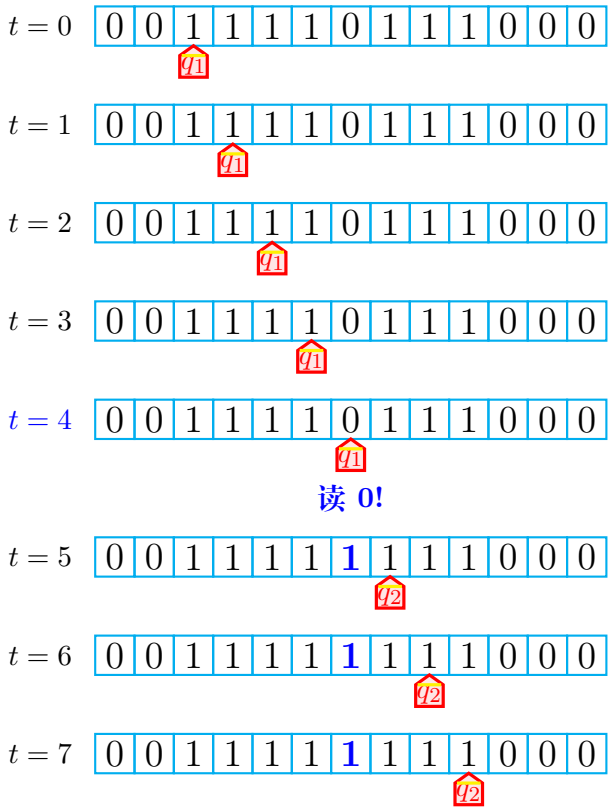


图 3: 图灵机执行过程（第一部分）：q_1 状态向右扫描，在 $t = 4$ 遇到 0 时写成 1 并转入 q_2

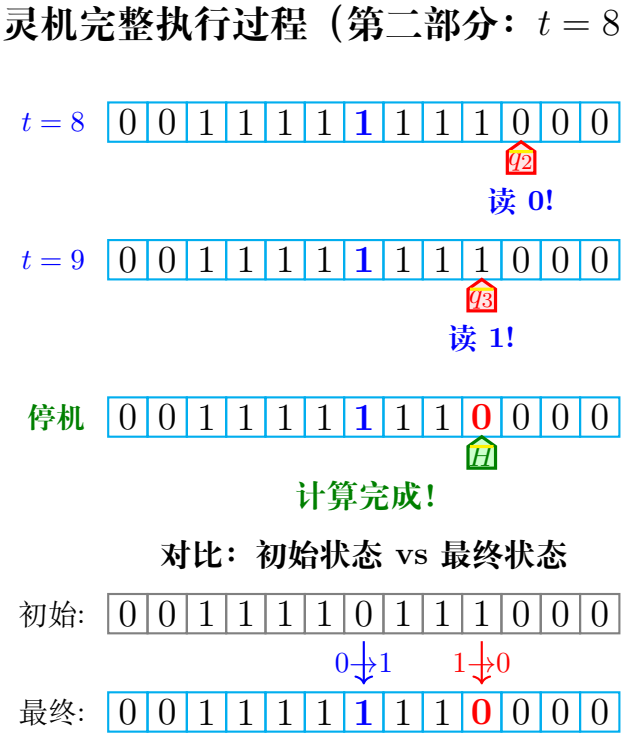


图 4: 图灵机执行过程（第二部分）：q_2 遇到 0 后左移进入 q_3，写 0 后停机

执行轨迹汇总

初始配置：纸带 `0011110111000`，读写头在位置 2（第一个 1），状态 q_1

步数 t	位置	状态	读到	动作
0	2	q_1	1	写 1, 右移, 保持 q_1
1	3	q_1	1	写 1, 右移, 保持 q_1
2	4	q_1	1	写 1, 右移, 保持 q_1
3	5	q_1	1	写 1, 右移, 保持 q_1
4	6	q_1	0	写 1, 右移, 转 q_2
5	7	q_2	1	写 1, 右移, 保持 q_2
6	8	q_2	1	写 1, 右移, 保持 q_2
7	9	q_2	1	写 1, 右移, 保持 q_2
8	10	q_2	0	写 0, 左移, 转 q_3
9	9	q_3	1	写 0, 停机

纸带变化：

- 位置 6：$0 \to 1$（在 $t = 4$ 时被 q_1 改写）
- 位置 9：$1 \to 0$（在 $t = 9$ 时被 q_3 改写）

最终纸带：`0011111110000`

直觉：从例子看图灵机的本质

观察：

1. **极其简单**：只有 3 个状态，6 条转移规则 2. **完全确定**：给定状态和符号，下一步动作唯一确定 3. **逐步执行**：每一步只做一件事：读、写、移动

Church-Turing 论题：

$$\boxed{\text{任何”可计算”的问题，都可以用图灵机来计算}}$$

等价表述：

- Python 程序能算的 ⇔ 图灵机能算的
- C++ 程序能算的 ⇔ 图灵机能算的
- 任何编程语言能算的 ⇔ 图灵机能算的

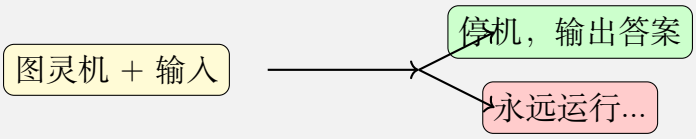
意义：图灵机是计算的”通用货币”——我们可以用它来定义”什么是可计算的”

5 图灵机一定会停机吗？

关键问题

观察：我们的例子运行了 10 步就停机了

但是：是不是所有图灵机都会停机？



5.1 停机问题

停机问题的定义

问题：给定图灵机 M 和输入 x，判断 M 在输入 x 上是否会停机

形式化：

$$\text{HALT}(M,x) = \begin{cases} 1 & \text{如果} M \text{ 在} x \text{ 上停机} \\ 0 & \text{如果} M \text{ 在} x \text{ 上永不停机} \end{cases}$$

问题：存在图灵机能计算 HALT 吗？

5.2 数学证明就是停机

哥德巴赫猜想的”图灵机”

哥德巴赫猜想（1742）：每个大于 2 的偶数都可以写成两个素数之和

例如：$4 = 2 + 2$，$6 = 3 + 3$，$8 = 3 + 5$，$10 = 3 + 7 = 5 + 5$，...

写成图灵机：

验证哥德巴赫猜想的程序

```python
def is_prime(n):
    if n < 2: return False
    for i in range(2, int(n**0.5) + 1):
        if n % i == 0: return False
    return True

def check_goldbach(n):
    """检查偶数 n 是否能写成两个素数之和"""
    for p in range(2, n):
        if is_prime(p) and is_prime(n - p):
            return True
    return False

# 枚举验证
n = 4
while True:
    if not check_goldbach(n):
        print(f"反例：{n}")
        break  # 停机！
    n += 2
# 如果没有反例，永远运行...
```

哥德巴赫猜想与停机问题

关键观察：

- 如果哥德巴赫猜想为**假** ⇒ 存在反例 ⇒ 程序会停机
- 如果哥德巴赫猜想为**真** ⇒ 没有反例 ⇒ 程序永不停机

推论：

$$\boxed{\text{证明哥德巴赫猜想} \Leftrightarrow \text{判定这个程序是否停机}}$$

但是：停机问题不可判定！
$\Rightarrow$ 可能存在**永远无法证明也无法证伪**的数学命题

黎曼猜想的"图灵机"

黎曼猜想（1859）：$\zeta(s)$ 的所有非平凡零点的实部都是 $\frac{1}{2}$

验证黎曼猜想的程序（概念性）

```python
def zeta_zeros():
    """生成 zeta 函数的非平凡零点"""
    # 用数值方法逐个找零点
    # ...

for zero in zeta_zeros():
    if abs(zero.real - 0.5) > epsilon:
        print(f"反例：{zero}")
        break  # 停机！
# 如果没有反例，永远运行...
```

同样的逻辑：
证明黎曼猜想 $\Leftrightarrow$ 判定这个程序是否停机

哲学：证明就是停机

数学证明的本质：
给定命题 P，"证否 P"等价于：

找到一个图灵机，它会停机当且仅当 P 为真

未解决的数学问题：
就是那些我们不知道对应图灵机是否停机的问题！

猜想	状态
费马大定理	已证明（1995）——程序不停机
四色定理	已证明（1976）——程序不停机
哥德巴赫猜想	未解决——不知道是否停机
黎曼猜想	未解决——不知道是否停机
P = NP?	未解决——不知道是否停机

数学猜想等价于多少状态的图灵机？

核心问题：验证一个数学猜想需要多复杂的图灵机?
思路：如果能把猜想编码成 n 状态的图灵机：

- 猜想为假 $\Leftrightarrow$ 存在反例 $\Leftrightarrow$ 图灵机停机
- 猜想为真 $\Leftrightarrow$ 无反例 $\Leftrightarrow$ 图灵机永不停机

Busy Beaver 函数 $\mathrm{BB}(n)$：n 状态图灵机从空白纸带能写的最多 1 的个数
关键性质：$\mathrm{BB}(n)$ 是**不可计算**的!
如果我们知道 $\mathrm{BB}(n)$，就可以运行任何 n 状态图灵机 $\mathrm{BB}(n)$ 步：

- 没停机 $\Rightarrow$ 永远不会停机 $\Rightarrow$ 对应的猜想为真
- 停机了 $\Rightarrow$ 找到反例 $\Rightarrow$ 对应的猜想为假

重要猜想的图灵机状态数

已知精确结果（Busy Beaver 挑战项目）：

n	$\mathrm{BB}(n)$	说明
1	1	
2	6	
3	21	
4	107	
5	$\mathbf{47,176,870}$	2024 年证明
6	$> 10^{10,000}$	未知精确值

含义：所有 5 状态图灵机要么在 47,176,870 步内停机，要么永不停机
$\Rightarrow$ 所有能用 ≤ 5 状态表达的数学问题都已”原则上可判定”!

经典猜想的状态估计

Collatz 猜想（$3n+1$ 问题）：
已被编码为 **15 状态**的图灵机（Aaronson & Yedidia, 2016）
$\Rightarrow$ 如果我们知道 $\mathrm{BB}(15)$，就能判定 Collatz 猜想!
其他猜想的估计：

猜想	状态数	来源/说明
Collatz 猜想	15 状态	Aaronson-Yedidia 构造
Goldbach 猜想	$\sim$ 20-25 状态	枚举偶数，检查素数分解
孪生素数猜想	$\sim$ 25-30 状态	枚举检查 $p, p+2$
黎曼猜想	$\sim$ 744 状态	Matiyasevich 估计（需要算术编码）

ZFC 一致性：约 **748 状态**

$\Rightarrow$ 如果知道 BB(748)，就能判定 ZFC 是否一致！

为什么这很重要？

Busy Beaver 的哲学意义：

1. 量化”证明的难度”

一个猜想需要 n 状态的图灵机来验证

$\Rightarrow$ 证明它至少需要知道 BB(n)

$\Rightarrow$ 这给出了证明难度的**下界**

2. 揭示数学的极限

BB(n) 不可计算 $\Rightarrow$ 存在”原则上不可证明”的数学命题

这是哥德尔不完备定理的另一种表述！

3. 简单规则的复杂性

我们的加法器	$\sim$ 25 状态
Collatz 猜想	15 状态，但 BB(15) 未知

15 个状态看起来比 25 个还简单，但我们无法判定它是否停机！

$\Rightarrow$ **状态数少不代表行为简单**

停机问题不可判定（详细版）

定理 5.1 (图灵，1936). *不存在图灵机能够判定任意图灵机在任意输入上是否停机。*

目标：证明不存在一个“万能预测器”H，满足

$$H(M,x) = \begin{cases} 1, & M(x) \text{ 会停机} \\ 0, & M(x) \text{ 不会停机} \end{cases}$$

Step 1：反证法假设

假设这样的图灵机 H 存在，并且预测永远正确。

Step 2：构造一个“反着来”的机器 D

定义新图灵机 D：

- 输入：图灵机 M 的编码
- 首先运行 $H(M, M)$（让程序以自己为输入）
- 如果 $H(M, M) = 1$（预测 $M(M)$ 会停机），则 D 进入死循环
- 如果 $H(M, M) = 0$（预测 $M(M)$ 不停机），则 D 立即停机

直观理解：

> D 总是故意与预测结果相反。

Step 3：关键问题——考察 $D(D)$
现在让 D 把自己作为输入：

$$D(D)$$

我们分析两种可能。

情况 A：假设 $D(D)$ 会停机
则根据 H 的定义：

$$H(D, D) = 1$$

但根据 D 的规则：

> 若 $H(D, D) = 1$，则 $D(D)$ 必须死循环。

矛盾。

情况 B：假设 $D(D)$ 不会停机
则：

$$H(D, D) = 0$$

但根据 D 的规则：

> 若 $H(D, D) = 0$，则 $D(D)$ 必须停机。

再次矛盾。

Step 4：结论
无论 $D(D)$ 是否停机都会产生矛盾。

因此最初假设错误：

$$\boxed{\text{不存在万能停机判定机}H}$$

□

核心思想（直觉版）

构造一个程序，使其行为永远与预测器相反，
从而让预测器在“自我预测”时必然失败。

启示：并非所有计算问题都能被算法解决，这为优化与学习算法设定了理论边界。

6 简单规则，复杂行为

Busy Beaver：简单规则的极限

Busy Beaver 问题：
对于 n 个状态的图灵机，从空白纸带开始，**能停机的**图灵机中，最多能写多少个 1?
记这个数为 $\mathrm{BB}(n)$（Busy Beaver 函数）
已知结果：

- $\mathrm{BB}(1) = 1$
- $\mathrm{BB}(2) = 6$
- $\mathrm{BB}(3) = 21$
- $\mathrm{BB}(4) = 107$
- $\mathrm{BB}(5) = 47,176,870$（2024 年由 bbchallenge 项目证明）
- $\mathrm{BB}(6) > 10^{10,000}$（超指数级大，未知精确值）

生命游戏：只有 4 条规则

Conway 生命游戏（1970）：
无限二维网格，每个格子”活”（黑）或”死”（白）
规则——数一数 8 个邻居中有多少是活的：

1. 活细胞，邻居 $< 2 \to$ 死（孤独死）

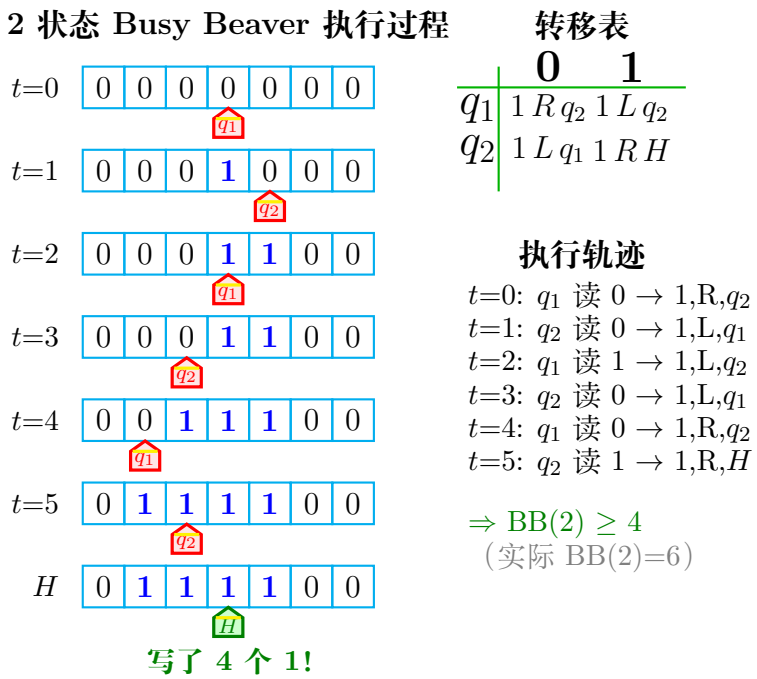


图 5: 2 状态 Busy Beaver：左侧为执行过程，右侧为转移表和执行轨迹

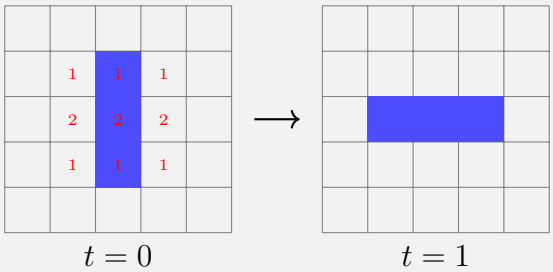
2. 活细胞，邻居 $= 2$ 或 $3 \to$ 活（维持）
3. 活细胞，邻居 $> 3 \to$ 死（拥挤死）
4. 死细胞，邻居 $= 3 \to$ 活（繁殖）

简化：

- 邻居数 $= 3 \Rightarrow$ 下一代**活**（不管现在死活）
- 邻居数 $= 2 \Rightarrow$ 下一代**保持**原状态
- 其他 $\Rightarrow$ 下一代**死**

例子：手算生命游戏一步

初始配置（Blinker，闪烁器）：



计算过程（红色数字是邻居数）：

- 格子 (1,2)：死细胞，邻居 =2 ⇒ 保持死
- 格子 (2,1)：活细胞，邻居 =1 ⇒ 孤独死
- 格子 (2,2)：活细胞，邻居 =2 ⇒ 维持活
- 格子 (2,3)：活细胞，邻居 =1 ⇒ 孤独死
- 格子 (3,2)：死细胞，邻居 =3 ⇒ **繁殖！**
- 格子 (1,2)：死细胞，邻居 =3 ⇒ **繁殖！**

结果：竖条变横条！再过一步又变回竖条（周期 =2）

Glider（滑翔机）：每 4 步向右下移动一格

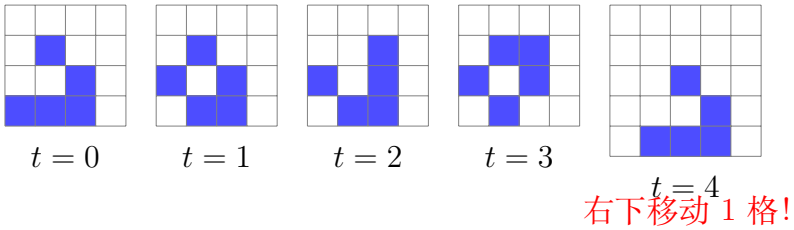


图 6: Glider 是生命游戏中最简单的”飞船”，永远向右下方移动

生命游戏是图灵完备的

定理（1982）：生命游戏可以模拟任意图灵机
意义：

- 只用 4 条规则，就能进行**任意计算**
- 给定初始配置，判断它是否会”稳定”是**不可判定**的

具体构造：

- 用 Glider 流表示”信号”（0 或 1）
- 用特殊结构实现”与门”、”或门”、”非门”
- 组合成完整的逻辑电路 ⇒ 可以模拟任何计算机

启示：最简单的规则也可能产生不可预测的复杂行为

Langton's Ant：混沌中的秩序

规则（只有 2 条！）：
一只"蚂蚁"在无限网格上行走：

- 站在**白格**上：把格子变黑，**右转** 90°，前进一步
- 站在**黑格**上：把格子变白，**左转** 90°，前进一步

Langton's Ant 前 8 步

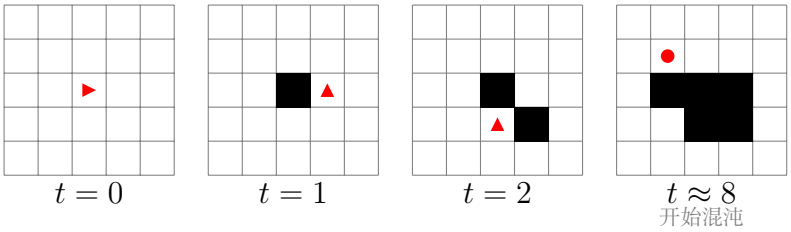
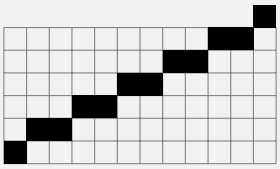


图 7: Langton's Ant 的前几步：看起来很简单...

Langton's Ant 的神秘行为

三个阶段：
阶段 1（约前 500 步）：简单对称图案
阶段 2（约 500-10000 步）：混沌！看起来完全随机
阶段 3（10000 步之后）：突然开始建造"高速公路"！



"高速公路"：104 步周期，斜向无限延伸

未解决的问题：

- 为什么会突然从混沌变成规则？没人知道！
- 是否**所有**初始配置最终都会产生高速公路？未证明！

关键：2 条规则，行为不可预测

Collatz 猜想：最简单的未解问题

规则（只有 1 条！）：
给定正整数 n，定义：

$$f(n) = \begin{cases} n/2 & \text{如果} n \text{ 是偶数} \\ 3n+1 & \text{如果} n \text{ 是奇数} \end{cases}$$

Collatz 猜想：从任何正整数开始，反复应用 f，最终都会到达 1

Collatz 猜想：具体计算

例 1：$n = 6$

$$6 \xrightarrow{\div 2} 3 \xrightarrow{\times 3+1} 10 \xrightarrow{\div 2} 5 \xrightarrow{\times 3+1} 16 \xrightarrow{\div 2} 8 \xrightarrow{\div 2} 4 \xrightarrow{\div 2} 2 \xrightarrow{\div 2} 1 \quad \checkmark$$

例 2：$n = 7$

$$7 \to 22 \to 11 \to 34 \to 17 \to 52 \to 26 \to 13 \to 40 \to 20 \to 10 \to 5 \to 16 \to 8 \to 4 \to 2 \to 1 \quad \checkmark$$

（17 步！）

例 3：$n = 27$（著名的”困难”例子）

- 需要 **111 步**才能到达 1
- 中途最大值达到 **9232**（比起始值大 342 倍！）

例 4：$n = 1$ 会陷入循环：$1 \to 4 \to 2 \to 1 \to \cdots$

为什么 Collatz 猜想这么难？

已验证范围：所有 $n \le 2^{68} \approx 3 \times 10^{20}$ 都会到达 1
但是：没有人能证明对**所有**正整数成立！
困难在哪里？

1. **不可预测性**：序列长度和最大值没有规律
2. **增长**：$3n+1$ 可能让数字变得很大
3. **没有不变量**：找不到一个在迭代中单调变化的量

图灵机编码：Collatz 猜想可以用 **15 状态**的图灵机表达
$\Rightarrow$ 如果我们知道 BB(15)，就能判定 Collatz 猜想！
Erdős 的评价：”Mathematics is not yet ready for such problems.”

核心结论：简单规则的教训

系统	规则数	复杂性
Busy Beaver (5 状态)	有限转移表	运行 47,176,870 步
生命游戏	4 条	图灵完备
Langton's Ant	2 条	混沌 → 秩序，未解释
Collatz 猜想	1 条	未证明

核心信息：

1. **简单不等于可预测**：规则简单不代表行为简单
2. **不能靠枚举**：即使规则有限，状态空间也可能无限
3. **计算的本质**：复杂性来自**迭代**，不是规则本身

7 哥德尔不完备性定理

不完备性定理

定理 7.1 (哥德尔，1931)**.** *任何足够强的一致的形式系统，都存在既不能证明也不能证伪的命题。*

通俗解释：

- "足够强"：能描述自然数的算术
- "一致"：不会同时证明 P 和 $\neg P$
- "不完备"：存在真命题无法在系统内证明

与停机问题的联系：
如果存在"万能"的证明系统，就可以判定停机问题
$\Rightarrow$ 这样的系统不存在

哲学：计算的边界

第一重边界：不可计算
有些问题**原则上**无法用算法解决（停机问题）
第二重边界：不可证明
有些真命题**原则上**无法在形式系统内证明（哥德尔）
对优化的启示：

1. 不可能有”万能优化算法”解决所有问题
2. 每个问题都有其**本质难度**
3. 我们需要针对**特定结构**设计方法

8 P 与 NP：可行计算的边界

从不可计算到”太慢”

好消息：有相当一部分实际问题是”可计算”的
坏消息：可计算不等于可以”快速”计算
例子：旅行商问题（TSP）

- 给定 n 个城市和距离，找最短环游路线
- 暴力枚举：$n!$ 种可能，$n = 20$ 时约 2.4×10^{18}
- 即使每秒检查 10^9 条路线，需要 76 年！

问题：什么是”快速”？什么是”慢”？

多项式时间：P 类

定义：算法的时间复杂度是 $O(n^k)$（k 是常数）
P 类：所有能在多项式时间内**求解**的决策问题
例子：

- 排序：$O(n \log n)$
- 最短路径：$O(n^2)$ 或 $O(n \log n)$
- 线性规划：$O(n^3)$
- 判断一个数是否为素数：$O(n^6)$（AKS 算法）

直觉：$O(n^k)$ 是”可行”的，$O(2^n)$ 是”不可行”的

n	n^2	n^3	2^n	$n!$
10	100	1000	1024	3.6×10^6
20	400	8000	10^6	2.4×10^{18}
50	2500	125000	10^{15}	3×10^{64}

验证比求解容易：NP 类

观察：有些问题很难求解，但验证答案很容易
例子：数独

- **求解**：可能需要大量回溯
- **验证**：给定填好的数独，检查是否合法只需 $O(n^2)$

NP 类：所有能在多项式时间内**验证**的决策问题
NP = Nondeterministic Polynomial（非确定性多项式时间）
显然：P $\subseteq$ NP（能求解的一定能验证）
千禧年问题：P = NP?

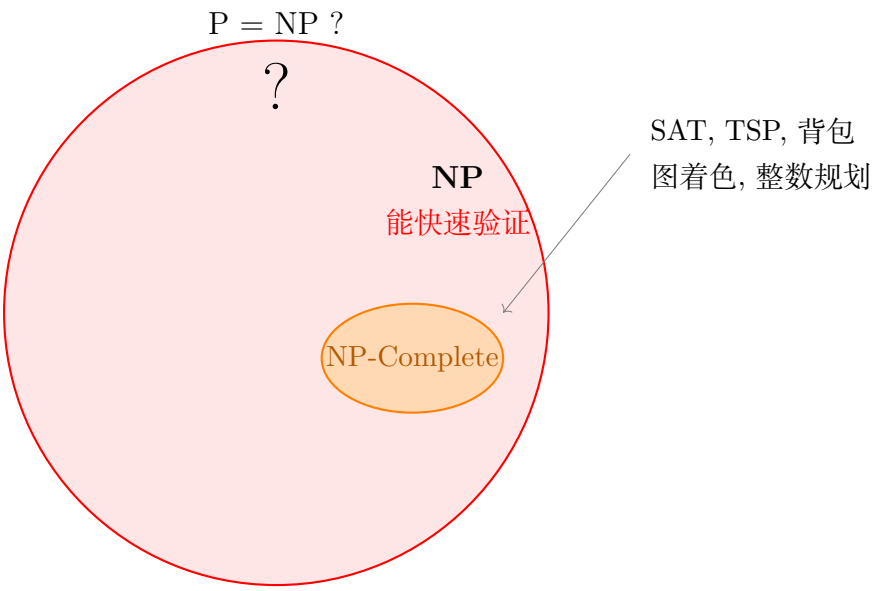


图 8: P、NP 和 NP-Complete 的关系

9 NP-Complete 与归约

NP-Complete：最难的 NP 问题

直觉：如果能快速解决一个 NP-Complete 问题，就能快速解决**所有** NP 问题
归约：把问题 A 转化为问题 B
如果 $A \leq_p B$（A 多项式时间归约到 B）：

- B 能快速解 $\Rightarrow$ A 能快速解
- A 很难 $\Rightarrow$ B 至少一样难

NP-Complete：NP 中最难的问题

- 在 NP 中（答案可以快速验证）
- 所有 NP 问题都能归约到它

什么是 SAT 问题？

布尔变量：只能取 True (1) 或 False (0)
布尔公式：用 $\wedge$（与）、$\vee$（或）、$\neg$（非）组合变量
SAT 问题：给定布尔公式，是否存在一组变量赋值使公式为真?
例子：

$$\phi = (x_1 \vee \neg x_2) \wedge (x_2 \vee x_3) \wedge (\neg x_1 \vee \neg x_3)$$

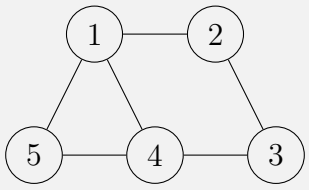
尝试 $x_1 = 1, x_2 = 1, x_3 = 0$：

- 子句 1：$(1 \vee \neg 1) = (1 \vee 0) = 1$
- 子句 2：$(1 \vee 0) = 1$
- 子句 3：$(\neg 1 \vee \neg 0) = (0 \vee 1) = 1$

$\Rightarrow$ 公式可满足！答案是 YES
3-SAT：每个子句恰好 3 个文字的 SAT（仍然是 NP-Complete）

什么是独立集问题？

独立集：图 $G = (V, E)$ 中的一个顶点子集 S，其中任意两点都**不相邻**
独立集问题：给定图 G 和整数 k，是否存在大小 $\geq k$ 的独立集?
例子：



问：是否有大小 ≥ 2 的独立集?

检验：$\{2, 5\}$ 是独立集吗?

- 2 和 5 之间有边吗？没有！

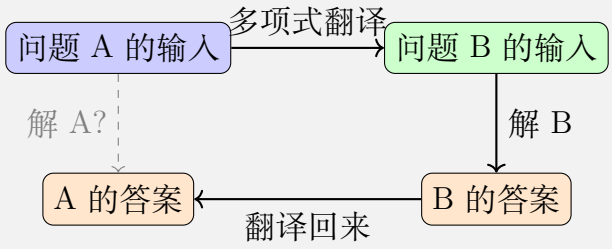
$\Rightarrow$ 是的，$\{2, 5\}$ 是大小为 2 的独立集
更大的：$\{2, 4, 5\}$ 是独立集吗?

- 4 和 5 之间有边！

$\Rightarrow$ 不是

归约的威力：问题之间的”翻译”

归约的含义：如果我们能把问题 A ”翻译”成问题 B：



如果翻译是多项式时间的：

- B 能快速解 $\Rightarrow$ A 也能快速解
- A 是 NP-Complete $\Rightarrow$ B 至少一样难

具体归约：3-SAT $\rightarrow$ 独立集

目标：把任意 3-SAT 公式”翻译”成一个独立集问题
输入：3-SAT 公式 ϕ，有 m 个子句，每个子句 3 个文字
翻译规则：

1. 每个子句创建 3 个顶点（对应 3 个文字），形成一个三角形
2. 三角形内部的 3 个顶点**两两连边**（子句内互斥）
3. 不同子句中，x_i 和 $\neg x_i$ 之间**连边**（矛盾互斥）

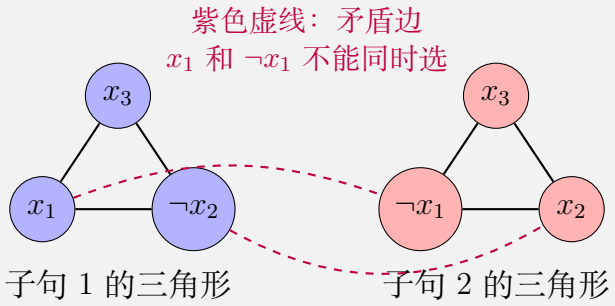
问：翻译后的图是否有大小 $= m$ 的独立集？

归约例子：手算一遍

3-SAT 公式：

$$\phi = (x_1 \vee \neg x_2 \vee x_3) \wedge (\neg x_1 \vee x_2 \vee x_3)$$

共 $m = 2$ 个子句
构造图：



问：是否有大小 $= 2$ 的独立集？

验证归约的正确性

独立集 $\{x_1, x_2\}$：

- 同一三角形内？x_1 在子句 1，x_2 在子句 2，不同
- 有矛盾边？x_1 和 x_2 之间没有边

$\Rightarrow$ 是大小为 2 的独立集！
对应的变量赋值：$x_1 = 1$，$x_2 = 1$（选中的文字为真）
验证 ϕ：

- 子句 1：$(1 \vee ? \vee ?) = 1 \quad (x_1 = 1)$
- 子句 2：$(? \vee 1 \vee ?) = 1 \quad (x_2 = 1)$

关键洞察：

- 三角形内只能选 1 个点 $\Leftrightarrow$ 每个子句至少选 1 个真文字
- 矛盾边阻止同时选 x_i 和 $\neg x_i$ $\Leftrightarrow$ 变量赋值一致

结论：ϕ 可满足 $\Leftrightarrow$ 图有大小为 m 的独立集

核心结论：归约的意义

3-SAT 是 NP-Complete（Cook-Levin 定理，1971）
我们刚才证明了：3-SAT $\leq_p$ 独立集
$\Rightarrow$ **独立集也是 NP-Complete！**
归约的威力：

- 不需要证明独立集”本身”很难
- 只需要把**已知困难**的问题翻译过来

- 这就是为什么 NP-Complete 问题" 一样难"

类似地可以证明：

$$\text{SAT} \to \text{3-SAT} \to \text{独立集} \to \text{团} \to \text{顶点覆盖} \to \cdots$$

10 为什么 SAT 能表达一切 NP 问题？

Cook-Levin 定理：计算 = 逻辑

定理 10.1 (Cook 1971, Levin 1973)**.** *SAT 是 NP-Complete 的。即：任何 NP 问题都可以多项式时间归约到 SAT。*

核心思想：图灵机的计算过程可以用布尔公式编码！

如何把计算编码成公式？

假设非确定性图灵机 M 在 T 步内判定问题，纸带长度为 T。
定义布尔变量（多项式个）：

- $\text{cell}_{t,i,s}$ = "时刻 t，位置 i 的符号是 s"
- $\text{head}_{t,i}$ = "时刻 t，读写头在位置 i"
- $\text{state}_{t,q}$ = "时刻 t，状态是 q"

用子句表达约束：

约束 1：每个位置恰好一个符号

至少一个符号：

$$\bigvee_{s\in\Gamma} \text{cell}_{t,i,s} \quad (\text{每个} t, i)$$

至多一个符号：对每对不同符号 $s \neq s'$

$$\neg\text{cell}_{t,i,s} \vee \neg\text{cell}_{t,i,s'}$$

类似地约束：每时刻恰好一个状态、读写头恰好在一个位置

约束 2：转移规则必须被遵守

如果转移函数 $\delta(q, a) = (q', b, R)$，则：

$$(\text{state}_{t,q} \wedge \text{head}_{t,i} \wedge \text{cell}_{t,i,a}) \Rightarrow (\text{state}_{t+1,q'} \wedge \text{cell}_{t+1,i,b} \wedge \text{head}_{t+1,i+1})$$

用 $A \Rightarrow B \equiv \neg A \vee B$ 转换成 CNF 形式
关键：每条转移规则对应若干子句，总共多项式个

约束 3 & 4：初始和终止条件

初始配置（输入 $x = x_1 x_2 \cdots x_n$）：

- $\text{state}_{0,q_{\text{start}}} = \text{True}$
- $\text{head}_{0,0} = \text{True}$
- $\text{cell}_{0,i,x_i} = \text{True}$（输入在纸带上）

终止条件：

$$\bigvee_{t=0}^{T} \text{state}_{t,q_{\text{accept}}}$$

完整公式 $\phi_M(x) =$ 所有约束的合取

直觉：SAT 的普适性

定理：M 接受 $x \Leftrightarrow \phi_M(x)$ 可满足
为什么？

- $\Rightarrow$：如果 M 接受 x，存在一条接受的计算路径，这条路径就是 $\phi_M(x)$ 的一个满足赋值
- $\Leftarrow$：如果 $\phi_M(x)$ 有满足赋值，这个赋值编码了 M 的一条有效计算路径，且到达接受状态

结论：SAT 足够强大，能**模拟任意计算**！
这就是为什么 SAT 是 “万能” 的 NP-Complete 问题。

10.1 3-SAT：恰到好处的复杂度

SAT 到 3-SAT 的归约

问题：Cook-Levin 证明的是 SAT 是 NP-Complete，为什么 3-SAT 也是？
答案：任意 SAT 公式都能转换成等价的 3-SAT 公式！

长子句的拆分

情况 1：子句只有 1 个文字 (x_1)
添加辅助变量：$(x_1 \vee y \vee z) \wedge (x_1 \vee y \vee \neg z) \wedge (x_1 \vee \neg y \vee z) \wedge (x_1 \vee \neg y \vee \neg z)$
情况 2：子句有 2 个文字 $(x_1 \vee x_2)$
添加辅助变量：$(x_1 \vee x_2 \vee y) \wedge (x_1 \vee x_2 \vee \neg y)$

长子句的拆分（续）

情况 3：子句有 $k > 3$ 个文字 $(x_1 \vee x_2 \vee \cdots \vee x_k)$
引入 $k-3$ 个辅助变量 $y_1, y_2, \ldots, y_{k-3}$：

$$
\begin{aligned}
&(x_1 \vee x_2 \vee y_1) \\
&(\neg y_1 \vee x_3 \vee y_2) \\
&(\neg y_2 \vee x_4 \vee y_3) \\
&\quad\vdots \\
&(\neg y_{k-3} \vee x_{k-1} \vee x_k)
\end{aligned}
$$

正确性：原子句为真 $\Leftrightarrow$ 新公式可满足

直觉：3-SAT 的归约正确性

直觉：辅助变量 y_i 像“传递棒”

- 如果 x_1, x_2 都为假，必须 $y_1 = 1$（才能满足第一个子句）
- 如果 $y_1 = 1$ 且 x_3 为假，必须 $y_2 = 1$
- $\cdots$ 一直传递下去
- 最终必须 x_{k-1} 或 x_k 为真（才能满足最后一个子句）

结论：SAT $\leq_p$ 3-SAT，所以 3-SAT 也是 NP-Complete

10.2 2-SAT：为什么突然变简单了？

2-SAT 可以多项式时间解决！

惊人事实：

- 3-SAT：NP-Complete（目前没有多项式算法）
- 2-SAT：$O(n+m)$ 时间可解！

关键差异：2-SAT 的子句 $(x \vee y)$ 等价于两个**蕴含**：

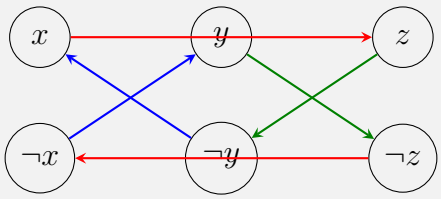
$$(x \vee y) \equiv (\neg x \Rightarrow y) \wedge (\neg y \Rightarrow x)$$

2-SAT 的图论视角

构造**蕴含图** G：

- 顶点：每个变量 x 和它的否定 $\neg x$
- 边：每个子句 $(a \vee b)$ 添加边 $\neg a \to b$ 和 $\neg b \to a$

例子：$(x \vee y) \wedge (\neg x \vee z) \wedge (\neg y \vee \neg z)$



2-SAT 的判定算法

定理：2-SAT 公式可满足 $\Leftrightarrow$ 蕴含图中不存在 x 和 $\neg x$ 在同一个强连通分量

为什么？

- 如果 x 和 $\neg x$ 在同一 SCC：存在路径 $x \to \cdots \to \neg x$ 和 $\neg x \to \cdots \to x$
- 这意味着 $x \Rightarrow \neg x$ 且 $\neg x \Rightarrow x$
- 无论 x 取什么值都矛盾！

算法：

1. 对蕴含图求强连通分量（Tarjan 算法，$O(n+m)$）
2. 检查是否有 x 和 $\neg x$ 在同一 SCC

直觉：为什么 3 是临界点？

2-SAT：每个子句是“二选一”，形成简单的蕴含链

- 结构太简单，用图的连通性就能解决

3-SAT：每个子句是“三选一”，可以编码“如果 A 且 B，则 C”

- 这种“与门”逻辑足以模拟图灵机的转移函数
- 能表达任意计算 $\Rightarrow$ NP-Complete

问题	复杂度	原因
2-SAT	P	只有蕴含，无法编码“与”
3-SAT	NP-Complete	能编码图灵机计算
k-SAT ($k \geq 3$)	NP-Complete	可归约到 3-SAT

教训：从 2 到 3，复杂度发生**相变**！

11 为什么图灵机是「计算」的标准？

Church-Turing 论题

论题（Thesis）：任何“直觉上可计算”的函数都可以被图灵机计算。
关键：这**不是**数学定理，**无法证明**！
为什么？

- “直觉上可计算”是非形式化的概念
- 图灵机是形式化的数学对象
- 我们无法证明一个非形式概念等于一个形式概念

但我们有强有力的**证据**支持这个论题...

证据：所有计算模型都等价

1936 年前后，多人独立提出“计算”的形式化定义：

模型	提出者	年份	风格
图灵机	Turing	1936	机械操作
λ 演算	Church	1936	纯函数变换
递归函数	Gödel, Kleene	1930s	数学归纳
Post 系统	Post	1936	字符串重写

惊人事实：这些完全不同的模型计算能力**完全相同**！

11.1 模型 1：图灵机

图灵机：机械打字员

隐喻：一个人拿着笔在无限长的纸带上做计算
核心操作：读、写、移动、改变状态
特点：

- 最直观，直接模拟人的计算过程
- 容易分析时间/空间复杂度
- 现代计算机硬件的理论原型

现代影响：计算机体系结构、复杂度理论

11.2 模型 2：λ 演算

λ 演算：一切皆函数

核心思想：计算 = 函数的“化简”
语法只有三条规则：

- 变量：x
- 抽象（定义函数）：$\lambda x.E$ 表示“输入 x，返回 E”
- 应用（调用函数）：$(F\ A)$ 表示“把 A 传给函数 F”

例子：

$$\begin{aligned} \text{恒等函数：} &\quad \lambda x.x \\ \text{常数函数：} &\quad \lambda x.\lambda y.x \\ \text{自应用：} &\quad (\lambda x.x\ x)(\lambda x.x\ x) \to \text{永不停止！} \end{aligned}$$

λ 演算的计算规则

β-归约：$(\lambda x.E)\ A \to E[x := A]$
把参数 A 代入函数体 E 中所有的 x

例子：

$$(\lambda x.x+1)\ 3 \to 3+1 \to 4$$

更复杂的例子：

$$(\lambda f.\lambda x.f(f(x)))\ (\lambda y.y+1)\ 0 \to (\lambda x.(x+1)+1)\ 0 \to 2$$

现代影响：

- Lisp (1958)：第一个函数式语言
- Haskell、ML、OCaml：纯函数式语言
- Python 的 `lambda`、JavaScript 的箭头函数

11.3 模型 3：递归函数

递归函数：从简单到复杂

核心思想：用有限规则从最简单的函数构造所有可计算函数
基本函数（公理）：

- 零函数：$Z(n) = 0$
- 后继函数：$S(n) = n + 1$
- 投影函数：$P_i^k(x_1, \ldots, x_k) = x_i$

构造规则：

- **复合**：$h(\vec{x}) = f(g_1(\vec{x}), g_2(\vec{x}), \ldots)$
- **原始递归**：从 f 和 g 构造 h
- μ **算子**：搜索使 $f(n) = 0$ 的最小 n

原始递归的例子：加法

定义 $\text{add}(m, n)$：

$$\text{add}(0, n) = n \quad \text{（基础情况）}$$
$$\text{add}(m+1, n) = S(\text{add}(m, n)) \quad \text{（递归：加 1）}$$

计算 $\mathrm{add}(2,3)$：

$$\begin{aligned}\mathrm{add}(2,3) &= S(\mathrm{add}(1,3))\\ &= S(S(\mathrm{add}(0,3)))\\ &= S(S(3))\\ &= S(4) = 5\end{aligned}$$

类似地可以定义乘法、指数、阶乘...
现代影响：递归算法的理论基础、可计算性理论

11.4 模型 4：Post 系统

Post 系统：字符串重写游戏

核心思想：计算 = 按规则改写字符串
形式：一组产生式规则 $\alpha \to \beta$
例子——一元加法（用 `1` 的个数表示数）：

规则 1： `11+1` $\to$ `1+11` （把一个 1 从左边移到右边）
规则 2： `1+` $\to \varepsilon$ （加号左边没有 1 了，删除）

计算 $2+3$（即 `11+111`）：

$$\begin{aligned}\texttt{11+111} &\to \texttt{1+1111} \quad \text{（规则 1）}\\ &\to \texttt{+11111} \quad \text{（规则 1）}\\ &\to \texttt{11111} \quad \text{（规则 2）}\end{aligned}$$

结果：5 个 `1`，即 5

Post 系统的现代影响

形式文法（Chomsky 层次）：

- 正则语言、上下文无关语言...
- 编程语言的语法定义

编译器：

- 词法分析、语法分析
- 代码变换和优化

其他应用：

- DNA 计算的理论模型
- 自动定理证明
- 项重写系统

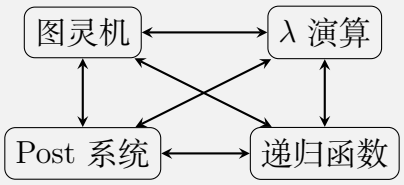
11.5 四种模型的统一

等价性定理

定理 11.1. *以下四种模型定义的“可计算函数”集合完全相同：*

1. *图灵机可计算的函数*
2. *λ 演算可定义的函数*
3. *递归函数*
4. *Post 系统可计算的函数*

证明思路：两两之间相互模拟



直觉：Church-Turing 论题的意义

更多证据：

- 现代计算机（C++、Python...）≡ 图灵机
- 量子计算机 ≡ 图灵机（只是某些问题更快）
- 80+ 年来没有找到反例

为什么图灵机“足够”？

1. **无限存储**：无限长纸带
2. **条件分支**：根据状态和符号决定行为
3. **循环**：可以回到之前的状态

这三点足以模拟任何算法的“读—判断—写—移动”过程。
结论：“可计算”有一个客观的、与模型无关的含义。无论你怎么定义“计算”，最终都会得到同样的边界！

四种风格总结

模型	隐喻	核心操作	现代影响
图灵机	机械打字员	读、写、移动	计算机硬件
λ 演算	数学函数	函数应用	函数式编程
递归函数	数学归纳	递归定义	递归算法
Post 系统	文字游戏	字符串替换	编译器、文法

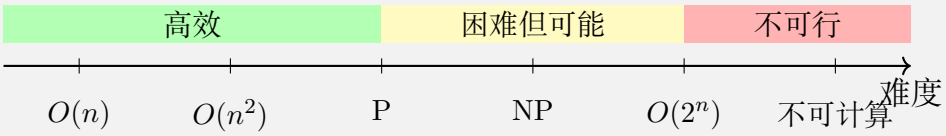
选择哪个模型？

- 分析复杂度 → 图灵机
- 编程语言设计 → λ 演算
- 可计算性理论 → 递归函数
- 形式语言 → Post 系统

殊途同归：它们回答的是同一个问题——什么是“计算”？

12 上限-下限框架

问题难度的光谱

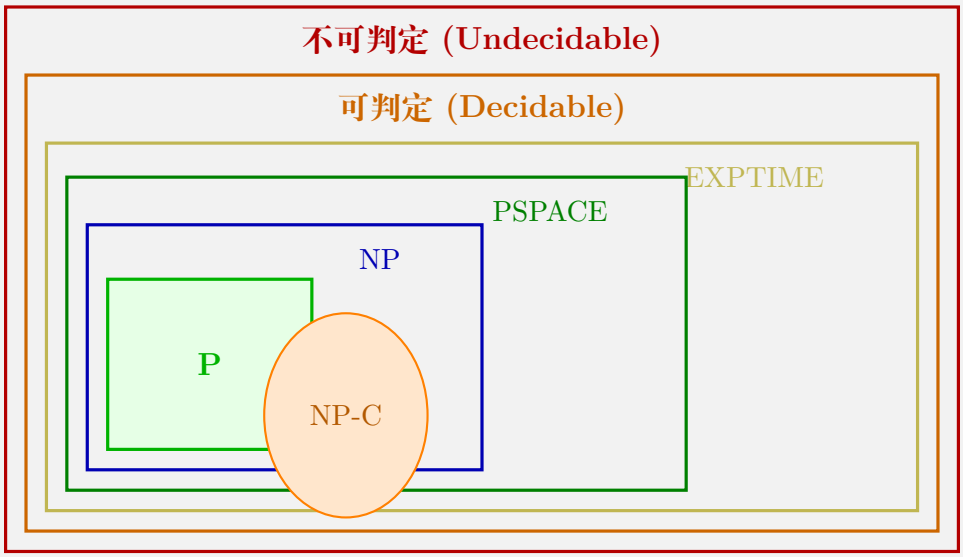


做研究的心态：

1. **先问上限**：问题有多难？是否 NP-Complete？是否不可计算？
2. **再问下限**：最坏情况下，我至少能做到什么？（暴力枚举？）
3. **寻找结构**：实际问题往往有特殊结构，可以利用

12.1 完整的复杂度层级

复杂度的完整光谱



包含关系：P ⊆ NP ⊆ PSPACE ⊆ EXPTIME ⊆ 可判定 ⊂ 所有问题

关键区别：NP-Complete vs 不可判定

	SAT (NP-Complete)	停机问题 (Undecidable)
有算法吗？	✓ 有！穷举 2^n 种赋值	× 不存在任何算法
能解决吗？	✓ 能，只是慢	× 永远不能
100 个变量？	2^{100} 步，很久但理论上能算	无论多少步都不行
本质	**有限的难**	**逻辑上不可能**

直觉：

- NP-Complete：超级复杂的密码锁，2^{100} 种组合，可以一个个试
- 不可判定：预测所有未来，存在自指悖论（“这句话是假的”）

12.2 各复杂度类的著名问题

P 类：多项式时间可解

定义：存在 $O(n^k)$ 算法的问题

问题	复杂度	算法
排序	$O(n \log n)$	快速排序、归并排序
最短路径	$O(V^2)$ 或 $O(E \log V)$	Dijkstra、Bellman-Ford
最大流	$O(V^2 E)$	Ford-Fulkerson
最大匹配	$O(V^3)$	匈牙利算法
线性规划	多项式时间	内点法
2-SAT	$O(n+m)$	强连通分量
欧拉回路	$O(E)$	Hierholzer 算法
最小生成树	$O(E \log V)$	Kruskal、Prim
凸包	$O(n \log n)$	Graham scan
矩阵乘法	$O(n^{2.37})$	Strassen 及改进

NP 类：可快速验证

定义：给定答案，能在多项式时间内验证

注意：P $\subseteq$ NP（能快速解的当然能快速验证）

问题	验证方式
SAT	代入赋值，检查每个子句 $O(m)$
图着色	检查每条边两端颜色不同 $O(E)$
哈密顿路径	检查路径访问每个顶点恰好一次 $O(V)$
子集和	计算子集的和是否等于目标 $O(n)$
整数分解	验证 $p \times q = n$ 只需一次乘法

核心问题：P = NP？能快速验证 $\Rightarrow$ 能快速求解？

NP-Complete：NP 中最难的问题

定义：在 NP 中，且所有 NP 问题都能归约到它

历史上第一个：SAT（Cook-Levin 定理，1971）

问题	描述
SAT / 3-SAT	布尔公式可满足性
团问题 (Clique)	图中是否有 k 个两两相连的顶点
独立集	图中是否有 k 个两两不相连的顶点
顶点覆盖	最少多少顶点能覆盖所有边
哈密顿回路	经过每个顶点恰好一次的回路
TSP（判定版）	是否存在长度 $\leq k$ 的旅行路线
图着色	能否用 k 种颜色着色使相邻不同色
子集和	是否存在子集使和等于目标值
背包问题	选物品使价值最大且重量不超限
整数线性规划	整数约束下的线性优化

关键：解决其中任何一个的多项式算法 $\Rightarrow$ P = NP

NP-Hard：至少和 NP-Complete 一样难

定义：所有 NP 问题都能归约到它，但它**不一定在 NP 中**

问题	类型	说明
TSP（优化版）	优化问题	求最短路线，不只是判定
停机问题	不可判定	比 NP 难得多！
围棋最优策略	EXPTIME-complete	指数时间
一般博弈	PSPACE 或更难	取决于博弈类型
非凸优化	NP-hard	找全局最优
神经网络训练	NP-hard	找最优权重

注意：NP-Hard $\not\subseteq$ NP，停机问题是 NP-Hard 但不在 NP 中

PSPACE：多项式空间

定义：可以用多项式大小的内存解决（时间可能是指数）

问题	说明
QBF (量化布尔公式)	$\forall x \exists y \ldots \phi(x, y, \ldots)$ 是否为真 PSPACE-complete
正则表达式等价	两个正则表达式是否匹配相同语言
两人有限博弈	一方是否有必胜策略
地理游戏	Generalized Geography
推箱子 (Sokoban)	给定局面能否通关

直觉：需要考虑“对手的所有可能回应”的问题

EXPTIME：指数时间

定义：需要 $O(2^{n^k})$ 时间的问题

问题	说明
围棋（$n \times n$ 棋盘）	EXPTIME-complete
国际象棋（广义）	EXPTIME-complete
跳棋（广义）	EXPTIME-complete
某些自动机等价性	判断两个自动机是否等价

注意：标准棋盘大小是固定的，这里说的是“广义”版本（$n \times n$）

不可判定：没有算法

定义：不存在图灵机能解决所有实例

问题	说明
停机问题	程序是否会停止
Post 对应问题	字符串匹配问题
Hilbert 第十问题	丢番图方程是否有整数解
矩阵死亡问题	矩阵乘积能否得到零矩阵
Rice 定理	程序的任何非平凡性质都不可判定
Kolmogorov 复杂度	最短描述长度不可计算
Wang Tiles	给定瓷砖能否铺满平面

关键：这些问题**永远**无法被算法解决，不是“慢”的问题

直觉：复杂度层级总结

复杂度类	时间	例子	实际意义
P	$O(n^k)$	排序、最短路	高效可解
NP	验证 $O(n^k)$	SAT、背包	暴力可解
NP-Complete	目前 $O(2^n)$	3-SAT、TSP	可能有结构
PSPACE	空间 $O(n^k)$	QBF、博弈	很难
EXPTIME	$O(2^{n^k})$	围棋	极难
不可判定	∞	停机问题	永远不能

实践启示：

- P 类：放心用，考虑常数优化
- NP-Complete：寻找问题结构，用启发式/近似算法

- 不可判定：换个问题问，或接受不完美的答案

核心结论：研究的本质

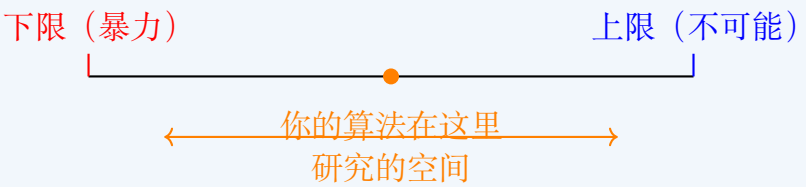
上限（Impossibility）：

- 不可计算：停机问题、哥德尔不完备
- NP-Complete：除非 P=NP，否则没有多项式算法

下限（Trivial Solution）：

- 暴力枚举：$O(2^n)$ 或 $O(n!)$
- 随机猜测
- 拉格朗日乘子法写出的 heuristic

科研本质：在上限和下限之间，发现问题的**结构**，开发**针对性方法**



优化问题的复杂度

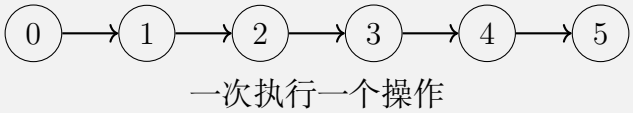
一般非凸优化：NP-hard
凸优化：多项式时间可解（本课程重点）
线性规划：多项式时间（单纯形法实际很快）
整数规划：NP-hard
神经网络训练：NP-hard（找全局最优），但实际中 SGD 效果很好
启示：

- 理论上的”困难”不意味着实际无法处理
- 实际问题往往有特殊结构
- 我们常常只需要”足够好”的解，而非最优解

13 GPU 改变复杂度观念

传统复杂度理论的假设

假设：计算是**顺序**执行的



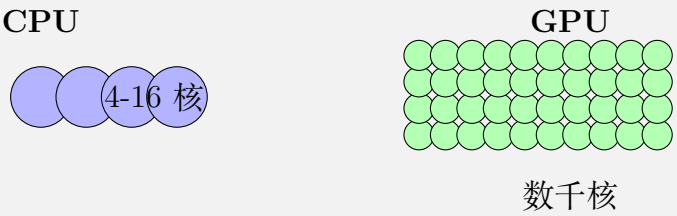
在这个假设下：

- $O(n \log n)$ 比 $O(n^2)$ 好
- 快速排序优于冒泡排序
- 复杂度越低越好

GPU：大规模并行

GPU 的特点：

- 数千个核心同时工作
- 适合”相同操作，不同数据”（SIMD）
- 内存带宽高，但延迟也高



新的规则：

- 并行度 > 复杂度（在一定规模内）
- 简单但可并行的算法可能更快

GPU 视角：复杂度悖论

冒泡排序 vs **快速排序**

	复杂度	并行性
冒泡排序	$O(n^2)$	高（奇偶排序可并行）
快速排序	$O(n \log n)$	低（分治有依赖）

实验结果（某些场景）：
当 $n < 10000$ 时，GPU 上的并行冒泡排序可能比 CPU 快速排序更快！
原因：

- GPU 可以同时比较数千对元素
- 快速排序的分治结构不利于 GPU

矩阵乘法：GPU 的杀手应用

问题：计算 $C = AB$，$A, B \in \mathbb{R}^{n \times n}$
朴素算法：$O(n^3)$ 次乘法
Strassen 算法：$O(n^{2.81})$（分治，减少乘法次数）
但是：GPU 上朴素算法更快！
原因：

- 朴素算法：n^2 个输出元素可以**完全并行**计算
- Strassen：递归分治，有数据依赖，难以并行

实际：cuBLAS 用朴素算法（及其变体），而非 Strassen

GPU 友好的特征

GPU 喜欢：

- 大量独立的简单计算
- 规则的内存访问模式
- 高算术强度（计算/内存比高）

GPU 不喜欢：

- 复杂的控制流（if-else 分支）
- 递归、动态数据结构
- 随机内存访问

对优化算法的启示：

- 矩阵运算天然适合 GPU
- SGD 的并行化（数据并行）
- 有些”高级”算法可能不如”朴素”算法

核心结论：第六条暗线：GPU 改变一切

传统观点：$O(n \log n)$ 绝对优于 $O(n^2)$
GPU 时代：

1. 并行度是关键因素
2. ”复杂度低”不等于”实际快”
3. 算法设计要考虑硬件特性

本课程的视角：
每个算法我们都会讨论：

- 理论复杂度
- 能否并行？怎么并行？
- GPU 上的实际表现

14 本周总结

哲学：面对困难问题的四步

第一步：意识到问题有多难
是和哥德巴赫猜想一样难（不可判定）？还是 NP-hard？还是多项式可解？
第二步：知道自己最差能做什么
暴力枚举 $O(2^n)$？拉格朗日乘子写个 heuristic？至少有个 baseline
第三步：寻找问题的特殊结构
没有万能算法。凸性？稀疏性？低秩？图结构？利用结构设计针对性方法
第四步：考虑硬件能提供什么
GPU 时代的算法设计：并行度、内存模式、计算/访存比

核心结论：第 1 周要点

1. **计算的形式化**
   - 图灵机：计算的”通用货币”
   - 算法 = 图灵机的转移规则
2. **计算的边界**
   - 停机问题不可判定
   - 数学证明 = 判定停机
   - 简单规则可以产生不可预测的行为
3. **复杂度层次**
   - P：能快速求解
   - NP：能快速验证
   - NP-Complete：NP 中最难的
4. **研究框架**
   - 上限：问题的本质难度
   - 下限：暴力方法
   - 中间：发现结构，设计方法
5. **GPU 视角**
   - 并行度 > 复杂度
   - 简单可并行算法可能更快

15 实验

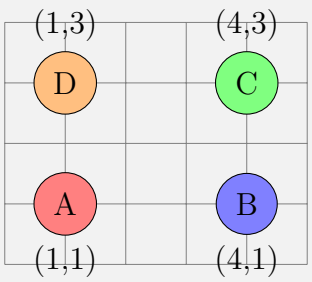
15.1 实验 1a：TSP - 结构的力量

什么是 TSP（旅行商问题）？

问题：一个旅行商要访问 n 个城市，每个城市恰好访问一次，最后回到起点。求**最短**的环游路线。
输入：n 个城市的坐标（或两两之间的距离矩阵）
输出：一个排列 $(c_1, c_2, \ldots, c_n)$ 表示访问顺序
目标：最小化总距离 $\sum_{i=1}^{n-1} d(c_i, c_{i+1}) + d(c_n, c_1)$

具体例子：4 个城市

城市坐标：



距离矩阵（欧氏距离）：

	A	B	C	D
A	0	3	$\sqrt{13} \approx 3.6$	2
B	3	0	2	$\sqrt{13} \approx 3.6$
C	3.6	2	0	3
D	2	3.6	3	0

问题：从 A 出发，访问 B、C、D 各一次，回到 A，最短路线是？

方法 1：暴力枚举（Brute Force）

思路：尝试**所有可能**的访问顺序，选最短的
4 个城市的所有排列（固定从 A 出发）：

路线	计算	总距离
A→B→C→D→A	$3+2+3+2$	$=10$ **最优**
A→B→D→C→A	$3+3.6+3+3.6$	$=13.2$
A→C→B→D→A	$3.6+2+3.6+2$	$=11.2$
A→C→D→B→A	$3.6+3+3.6+3$	$=13.2$
A→D→B→C→A	$2+3.6+2+3.6$	$=11.2$
A→D→C→B→A	$2+3+2+3$	$=10$ **最优**

最优路线：A→B→C→D→A 或 A→D→C→B→A（对称），总距离 $=10$
时间复杂度：$(n-1)! = O(n!)$
$n=10$: $9! = 362,880$ 种
$n=15$: $14! \approx 8.7 \times 10^{10}$ 种（不可行！）

方法 2：贪心算法（Greedy）

思路：每一步都选择**当前最近**的未访问城市
执行过程（从 A 出发）：

1. 当前在 A，未访问：{B, C, D}

   距离：A→B=3, A→C=3.6, A→D=2

   选最近的：**D**（距离 2）

2. 当前在 D，未访问：{B, C}

   距离：D→B=3.6, D→C=3

   选最近的：**C**（距离 3）

3. 当前在 C，未访问：{B}

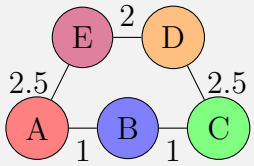
   只能选 **B**（距离 2）

4. 回到 A（距离 3）

贪心路线：A→D→C→B→A
总距离：$2+3+2+3=10$ 恰好是最优！
但是：贪心不保证最优，可能陷入局部最优
时间复杂度：$O(n^2)$（每步检查所有未访问城市）

贪心可能失败的例子

5 个城市（精心构造）：



贪心（从 A 出发）：

- A→B（最近，距离 1）
- B→C（距离 1）
- C→D（距离 2.5）
- D→E（距离 2）
- E→A（距离 2.5）

总距离：$1 + 1 + 2.5 + 2 + 2.5 = 9$
最优：A→E→D→C→B→A $= 2.5 + 2 + 2.5 + 1 + 1 = 9$（相同）
或 A→B→C→D→E→A $= 1 + 1 + 2.5 + 2 + 2.5 = 9$（相同）
更复杂的例子：贪心可能比最优差 20% 以上

方法 3：2-opt 局部优化

核心思想：给定一条路线，尝试**翻转**其中一段，看是否变短
2-opt 操作：

原始：A→B→C→D→E→A 优化后：A→B→D→C→E→A
红色部分交叉了！ 消除了交叉！

2-opt 的含义：选择两条边 $(i, i+1)$ 和 $(j, j+1)$，把中间的部分**翻转**

2-opt 具体例子

原始路线：A→B→C→D→A（总距离假设为 12）
尝试所有 2-opt：

1. 翻转 [B,C]：A→C→B→D→A，计算新距离
2. 翻转 [C,D]：A→B→D→C→A，计算新距离
3. 翻转 [B,C,D]：A→D→C→B→A，计算新距离

如果某次翻转使总距离变短，就**接受**这次改变，继续尝试
终止条件：没有任何 2-opt 操作能改进 ⇒ 达到**局部最优**
时间复杂度：每轮 $O(n^2)$，通常需要 $O(n)$ 轮 ⇒ $O(n^3)$
实际效果：贪心 + 2-opt 通常能得到接近最优的解！

三种方法对比

方法	复杂度	保证最优？	实用性
暴力枚举	$O(n!)$	是	$n \leq 12$
贪心	$O(n^2)$	否	快但可能差
贪心 + 2-opt	$O(n^3)$	否	通常很好

关键洞察：

- 暴力枚举是**上界**——保证正确但太慢
- 贪心是**启发式**——快但可能陷入局部最优
- 2-opt 利用**局部结构**——通过小改动逐步改进

这就是"发现结构"的力量：不是搜索所有可能，而是利用问题的几何性质

实验：TSP 的不同算法

TSP 三种算法实现

```python
import numpy as np
from itertools import permutations
import time

def generate_cities(n, seed=42):
    """生成 n 个随机城市的坐标"""
```

```python
    np.random.seed(seed)
    return np.random.rand(n, 2) * 100  # 100x100 的区域

def total_distance(tour, cities):
    """计算环游总距离"""
    n = len(tour)
    dist = 0
    for i in range(n):
        dist += np.linalg.norm(cities[tour[i]] - cities[tour[(i+1)%n]])
    return dist

# ===== 方法1: 暴力枚举 O(n!) =====
def tsp_brute_force(cities):
    """尝试所有排列，返回最短的"""
    n = len(cities)
    best_tour = None
    best_dist = float('inf')

    # 枚举所有排列
    for tour in permutations(range(n)):
        d = total_distance(tour, cities)
        if d < best_dist:
            best_dist = d
            best_tour = tour
    return list(best_tour), best_dist

# ===== 方法2: 贪心算法 O(n^2) =====
def tsp_greedy(cities):
    """每次选最近的未访问城市"""
    n = len(cities)
    visited = [False] * n
    tour = [0]  # 从城市0出发
    visited[0] = True

    for _ in range(n - 1):
        last = tour[-1]
        best_next = None
```

```python
        best_dist = float('inf')
        # 找最近的未访问城市
        for j in range(n):
            if not visited[j]:
                d = np.linalg.norm(cities[last] - cities[j])
                if d < best_dist:
                    best_dist = d
                    best_next = j
        tour.append(best_next)
        visited[best_next] = True

    return tour, total_distance(tour, cities)

# ===== 方法3: 2-opt 局部优化 =====
def tsp_2opt(cities, initial_tour=None):
    """2-opt:␣尝试翻转路线的一段来改进"""
    n = len(cities)
    if initial_tour is None:
        tour = list(range(n))
    else:
        tour = list(initial_tour)

    improved = True
    while improved:
        improved = False
        for i in range(n - 1):
            for j in range(i + 2, n):
                # 尝试翻转 tour[i+1:j+1]
                new_tour = tour[:i+1] + tour[i+1:j+1][::-1] +
                    tour[j+1:]
                if total_distance(new_tour, cities) <
                    total_distance(tour, cities):
                    tour = new_tour
                    improved = True
    return tour, total_distance(tour, cities)
```

实验：运行对比实验

对比三种方法

```python
# ===== 对比实验 =====
for n in [8, 10, 12]:
    cities = generate_cities(n)
    print(f"\n=== {n} 个城市 ===")

    # 暴力枚举
    start = time.time()
    _, dist_bf = tsp_brute_force(cities)
    time_bf = time.time() - start
    print(f"暴力枚举: 距离={dist_bf:.2f}, 时间={time_bf:.4f}s")

    # 贪心
    start = time.time()
    tour_g, dist_g = tsp_greedy(cities)
    time_g = time.time() - start
    gap_g = (dist_g / dist_bf - 1) * 100
    print(f"贪心:     距离={dist_g:.2f}, 时间={time_g:.4f}s, 误差={gap_g:.1f}%")

    # 贪心 + 2-opt
    start = time.time()
    _, dist_2opt = tsp_2opt(cities, tour_g)
    time_2opt = time.time() - start
    gap_2opt = (dist_2opt / dist_bf - 1) * 100
    print(f"贪心+2opt: 距离={dist_2opt:.2f}, 时间={time_2opt:.4f}s, 误差={gap_2opt:.1f}%")
```

运行结果（示例）：

```
=== 8 个城市 ===
暴力枚举: 距离=234.52, 时间=0.0312s
贪心:     距离=251.38, 时间=0.0001s, 误差=7.2%
贪心+2opt: 距离=234.52, 时间=0.0008s, 误差=0.0%

=== 10 个城市 ===
暴力枚举: 距离=267.13, 时间=2.8451s
贪心:     距离=289.45, 时间=0.0001s, 误差=8.4%
```

```
贪心+2opt: 距离=267.13, 时间=0.0015s, 误差=0.0%

=== 12 个城市 ===
暴力枚举: 距离=298.71, 时间=354.21s  # 约6分钟!
贪心:     距离=321.08, 时间=0.0002s, 误差=7.5%
贪心+2opt: 距离=298.71, 时间=0.0023s, 误差=0.0%
```

核心结论：TSP 实验的启示

观察：

- $n = 12$ 时暴力需要 6 分钟，$n = 15$ 需要数小时，$n = 20$ 不可行
- 贪心瞬间完成，但有 7-8% 的误差
- 贪心 + 2-opt 几乎瞬间完成，且经常找到最优解！

核心信息：

利用问题结构（局部优化）比**暴力搜索**高效得多
这就是优化算法研究的核心：发现并利用结构

思考：为什么 2-opt 这么有效？
因为好的 TSP 解通常**没有交叉**，而 2-opt 正好能消除交叉！

15.2 实验 1b：GPU 并行 vs CPU 顺序

实验：矩阵乘法：NumPy vs CuPy

```python
import numpy as np
import time

# 如果有 GPU, 导入 CuPy
try:
    import cupy as cp
    HAS_GPU = True
except ImportError:
    HAS_GPU = False
    print("CuPy not available, skipping GPU tests")

def benchmark_matmul(n, device='cpu'):
```

```python
    """测试矩阵乘法性能"""
    if device == 'cpu':
        A = np.random.randn(n, n).astype(np.float32)
        B = np.random.randn(n, n).astype(np.float32)

        # 预热
        _ = A @ B

        start = time.time()
        for _ in range(10):
            C = A @ B
        elapsed = (time.time() - start) / 10

    else:  # GPU
        A = cp.random.randn(n, n, dtype=cp.float32)
        B = cp.random.randn(n, n, dtype=cp.float32)

        # 预热
        _ = A @ B
        cp.cuda.Stream.null.synchronize()

        start = time.time()
        for _ in range(10):
            C = A @ B
        cp.cuda.Stream.null.synchronize()
        elapsed = (time.time() - start) / 10

    gflops = 2 * n**3 / elapsed / 1e9
    return elapsed, gflops

# 对比测试
sizes = [256, 512, 1024, 2048, 4096]

print("=" * 50)
print(f"{'Size':>8} | {'CPU (s)':>10} | {'GPU (s)':>10} | {'Speedup':>8}")
print("=" * 50)

```

```python
for n in sizes:
    cpu_time, cpu_gflops = benchmark_matmul(n, 'cpu')

    if HAS_GPU:
        gpu_time, gpu_gflops = benchmark_matmul(n, 'gpu')
        speedup = cpu_time / gpu_time
        print(f"{n:>8} | {cpu_time:>10.4f} | {gpu_time:>10.4f} | {speedup:>8.1f}x")
    else:
        print(f"{n:>8} | {cpu_time:>10.4f} | {'N/A':>10} | {'N/A':>8}")
```

典型输出：

```
==================================================
    Size |    CPU (s) |    GPU (s) |   Speedup
==================================================
     256 |     0.0012 |     0.0001 |      12.0x
     512 |     0.0089 |     0.0002 |      44.5x
    1024 |     0.0654 |     0.0005 |     130.8x
    2048 |     0.4821 |     0.0032 |     150.7x
    4096 |     3.7234 |     0.0245 |     151.9x
```

观察：GPU 比 CPU 快 100-150 倍！

实验：复杂度悖论：简单算法在 GPU 上的优势

```python
# 概念演示：并行前缀和 vs 顺序前缀和

def sequential_prefix_sum(arr):
    """顺序前缀和 O(n)，但完全串行"""
    n = len(arr)
    result = np.zeros(n)
    result[0] = arr[0]
    for i in range(1, n):
        result[i] = result[i-1] + arr[i]  # 必须等待前一个
    return result

def parallel_prefix_sum_concept(arr):
    """并行前缀和（Blelloch算法）的概念
```

```

    总操作数 O(n)，但可以在 O(log n) 步内完成
    每一步有很多独立操作可以并行
    """
    # Up-sweep (reduce)
    # Down-sweep
    # 详细实现略...
    pass

# 关键信息：
# - 顺序前缀和：n-1 步，每步 1 个操作
# - 并行前缀和：log(n) 步，每步可以有 n/2 个并行操作

# 在 GPU 上：
# - 顺序算法：无法利用并行性
# - 并行算法：可以利用数千核心

print("关键洞察：")
print("- 顺序算法 O(n)：n 步，无法并行")
print("- 并行算法 O(n)：log(n) 步 × n/2 并行操作")
print("- GPU 有数千核心 -> 并行算法实际更快！")
```

核心结论：实验总结

实验 1a：

- NP-hard 问题的暴力解法很快爆炸
- 利用**结构**（贪心 + 局部优化）可以高效求解
- ”足够好”的解往往比”最优解”更实际

实验 1b：

- GPU 在矩阵运算上比 CPU 快 100+ 倍
- 并行度是关键因素
- 算法设计要考虑硬件特性

16 思考题

思考题

1. 设计一个图灵机，判断输入的二进制数是否为偶数。（提示：只需要看最后一位）

2. 以下哪些问题是 NP-Complete 的？

(a) 判断一个图是否有欧拉回路

(b) 判断一个图是否有哈密顿回路

(c) 找到图的最大匹配

(d) 找到图的最大独立集

3. 为什么 GPU 上的矩阵乘法不用 Strassen 算法？

4. 凸优化问题是 P 还是 NP？为什么这对深度学习很重要？

5. 设计一个实验，比较在你的电脑上 CPU 顺序排序和 GPU 并行排序的性能。在什么规模下 GPU 开始占优势？